

SWIFTCHOICE

Systems Requirements Specification (SRS) & Architecture Baseline

Author: Tracy Nguyen | Business Analyst

Project Scope: Decision-Support Mobile Application MVP

Version: 1.0

Date: 2026

Table of Contents

1. Problem Statement & Objectives	1
2. Scope	1
3. Customer Journey (Food module)	2
4. Persona	3
5. Elicited Requirements	3
6. User Story	4
US-01: Food Single Meal Recommendation	4
US-02: Food 1-Time Decision Reroll	5
US-03: Focus Workspace Matching	6
US-04: Focus 1-Time Decision Reroll	7
US-05: Task Input & Weighted Matrix Priority Calculation	8
US-06: Priority Task Completion & Queue Transition	8
US-07: User Authentication	9
US-08: Decision History Log	10
7. Key Design Considerations	10
8. Requirements Traceability Matrix	11
9. Process Diagram	13
Use Case Diagram	13
Activity Diagram	14
ER Diagram	17
Data Dictionary	17
Sequence Diagram	22
10. UAT Test Cases	23
Test Case: TC-FOOD-01a	23
Test Case: TC-FOOD-01b	23
Test Case: TC-FOOD-02	23
Test Case: TC-FOC-01	24
Test Case: TC-FOC-02	24
Test Case: TC-PRI-01	24
Test Case: TC-PRI-02	25
Test Case: TC-AUTH-01	25
Test Case: TC-HIST-01	25
11. Future Enhancements	26

1. Problem Statement & Objectives

SwiftChoice addressed decision fatigue - the cognitive exhaustion from making too many choices (Schwartz & Schuldenfrei 2025). This is not just theoretical: 82% of Australian consumers abandon a decision when information becomes too complex (CDOTrends 2023), and primary survey data (n=21) found 47.6% of respondents feel overwhelmed “often” or “very often”, with 38% spending over 20 minutes deliberating a single choice.

Despite this, only 4.8% currently use a decision-making app - a clear gap between problem scale and available solutions. SwiftChoice targets this gap for routine, low-stakes decisions (what to eat, where to work, what to prioritise), rather than the major life decisions already well-served by existing tools.

Objective: rather than presenting more options, SwiftChoice narrows each decision to a single, personalised recommendation, addressing meal choice (Food), workspace choice (Focus), and task prioritisation (Priority), with a one-time reroll if the suggestion does not fit.

2. Scope

Scope Statement: This document defines the requirements for the SwiftChoice MVP - a mobile decision-support application covering three independent decision modules. Each module operates standalone; modules do not reference or depend on one another.

In Scope (MVP):

Area	Included
Food module	Single meal/restaurant recommendation based on budget, prep time, distance; one-time reroll per session
Focus module	Single study/work spot recommendation based on energy level and environment vibe; one-time reroll per session; weather shown as reference only
Priority module	Task input with manual Importance/Urgency tagging; weighted-matrix ranking; task completion tracking (no reroll - deterministic output)

Supporting features	User authentication, decision history logging, weekly Mental Energy dashboard, local-first preference storage
---------------------	---

Out of Scope (MVP):

- Payment / subscription tier (SwiftChoice Pro) - commercial model proposed for a later phase, not part of MVP
- AI-assisted prioritisation for the Priority module - proposed enhancement, requires further scoping
- Weather-influenced matching algorithm for Focus - currently informational display only
- Social or group decision-making features
- Web companion application (mobile-only for MVP)

3. Customer Journey (Food module)

Stage	User thoughts / actions	Pain points	Evidence
Awareness	Realises they need to eat but doesn't know what to choose - too many options come to mind (restaurants, cooking, delivery apps).	Decision fatigue compounds - food choice becomes another burden, not a pleasure	47.6% feel overwhelmed "often/very often" when making decisions
Consideration	Mentally runs through options, weighs budget and time available, starts second-guessing	Overthinking; anxiety about choosing "wrong"	Survey quote: "Overthink and worry about making the wrong choice"; 38% spend 20+ minutes deliberating a single choice
Decision	Asks friends/family, does a Google search, browses social media for ideas	Fragmented process across multiple channels; still lacks confidence in the result	52.4% rely on friends/family and 52.4% rely on Google Search to help decide

Action	Finally settles on an option - often defaulting to a familiar/repetitive choice to end the deliberation	Time and mental energy already spent before the meal is even chosen; low satisfaction with the process itself	Survey quote: "I feel stressed when I have to make decisions quickly"
--------	---	---	---

4. Persona

User Persona

A profile of a target SwiftChoice user, built from survey research (n=21), informing module design and requirements.

Core Profile	Background	Audience Insights
 Name: Anna Age: 25 Job: Part-time barista, full-time student Income range: Under \$30,000/year Location: Melbourne, AU	Short bio: Anna juggles university coursework with part-time work. Between classes and shifts, she has little time or energy left to plan meals, so she often ends up eating the same thing repeatedly or ordering delivery out of decision fatigue. Lifestyle and values: Wants to eat well and stay within budget, but prioritises convenience over meal planning; values quick, low-effort decisions given her limited free time. Professional context: Studies during the day, works evening/weekend shifts - irregular schedule makes routine meal prep difficult.	Goals: - Short-term: Decide what to eat quickly, without spending 20+ minutes deliberating - Long-term: Build a healthier, more consistent eating routine without extra mental effort - Key drivers and priorities: - Speed - Budget - Low cognitive effort Barriers: - Practical challenges: Limited time between class/work; tight budget - Emotional frustrations: Overthinks choices, worries about "wasting" money or time on the wrong pick - Obstacles to success: Too many options across apps/platforms, no single trusted source to decide quickly

5. Elicited Requirements

- User wants to receive a single recommendation, not a list, to avoid comparing multiple options
- User wants clear details like address to act on the recommendation immediately
- User wants confidence that the recommended venue is open/available at the time they plan to eat
- User wants the recommendation to correctly match the constraints they chose (budget, time, distance)

- User wants the whole process to be quick and low-effort, fitting into their limited free time

6. User Story

US-01: Food Single Meal Recommendation

- User Story: As a busy individual, I want a system to suggest a single meal option based on my budget, preparation time, and distance, so that I can make an eating decision without experiencing choice overload.
- Acceptance Criteria:
 1. Scenario 1a: Successful Meal Recommendation - Eat Out
 - GIVEN the user is on the Food Input screen and has selected "Eat Out", and has set a budget range, prep time, and distance constraints,
 - WHEN the user taps "Decide For Me",
 - THEN the system searches for venues matching constraints, fetches full details for one randomly selected venue from the list and displays it on the Food Result screen within 2.0 seconds, showing the venue's name, address, price tier, distance, and an explanation label, with a compact summary line confirming the applied filters.
 2. Scenario 1b: Successful Meal REcommendation - Eat In
 - GIVEN the user is on the Food Input screen, has selected "Eat In", and has set a budget range and prep time (distance is not applicable),
 - WHEN the user taps "Decide for Me",
 - THEN the system queries a recipe/meal suggestion matching the constraints and displays it on the Food Result screen within 2.0 seconds, showing the meal name, price tier, prep time, and an explanation label, with a compact summary line confirming the applied filters.
 3. Scenario 2a: Validation on Incomplete Constraints - Eat Out
 - GIVEN the user is on the Food Input screen with "Eat Out" selected,
 - WHEN the user attempts to tap "Decide For Me" without selecting mandatory constraints (budget, prep time, and distance),
 - THEN the system prevents submission and highlights the missing fields with an inline prompt.
 4. Scenario 2b: Validation on Incomplete Constraints - Eat In
 - GIVEN the user is on the Food Input screen with "Eat In" selected,
 - WHEN the user attempts to tap "Decide for Me" without selecting mandatory constraint (budget and prep time),
 - THEN the system prevents submission and highlights the missing fields with an inline prompt.
 5. Scenario 3: Zero Matching Options
 - GIVEN no matching options are found for the selected constraints (no venues within range for Eat Out, or no recipes matching for Eat In),
 - WHEN the user taps "Decide for Me",
 - THEN the system displays: "No options found matching your criteria. Try expanding your search radius or adjusting your budget." and return the user to the input selection screen with inputs retained.

6. Scenario 4: Network Connectivity Lost (Eat Out)

- GIVEN the user is on the Food Input screen with “Eat Out” selected and no active internet connection,
- WHEN the user taps “Decide For Me”,
- THEN the system displays: “No internet connection. Please check your network and try again.” and keeps the “Decide For Me” button enabled for retry.

7. Scenario 5: External API Failure (Eat Out)

- GIVEN the user has “Eat Out” selected and an active internet connection but the Places API fails to respond or times out,
- WHEN the user taps “Decide For Me”,
- THEN the system displays: “Something went wrong while finding your options. Please try again in a moment.” and offers a “Retry” action.

8. Scenario 6: Recommendation Filtering by Operating Hours

- GIVEN the user is on the Food Input screen within “Eat Out” selected and has set budget, prep time, and distance constraints,
- WHEN the user taps “Decide For Me”,
- THEN the system only considers venues that are open at the user’s requested eating time when generating the recommendation, excluding any venue that is closed at that time from the result pool.

US-02: Food 1-Time Decision Reroll

- User Story: As a user dissatisfied with the initial suggestion, I want to reroll the recommendation once per session, so that I have a quick alternative option without getting trapped in endless decision-making.
- Acceptance Criteria:
 1. Scenario 1: Successful First Reroll
 - GIVEN the initial suggestion is displayed, the candidate list contains at least one more unfetched venue, and session reroll count equals 0,
 - WHEN the user taps the “Reroll” button,
 - THEN the system fetches details for the next venue in the candidate list and displays it within 2.0 seconds and increments the reroll counter to 1.
 2. Scenario 2: Enforcing Reroll Limit
 - GIVEN the user has already used their one allowed reroll in the active session,
 - WHEN the alternative suggestion is displayed,
 - THEN the “Reroll” button is disabled (greyed out) and displays a message: “No rerolls remaining for this session.”
 3. Scenario 3: Resetting Reroll State on New Session
 - GIVEN a completed Food session where a reroll occurred,
 - WHEN the user returns to the Home screen and initiates a fresh Food request,
 - THEN the system resets the session reroll counter back to 0.
 4. Scenario 4: Network Connectivity Lost
 - GIVEN the user is on the Food Result screen with no active internet connection suddenly,
 - WHEN the user taps “Reroll”,
 - THEN the system displays: “No internet connection. Please check your network and try again.”, does not increment the reroll counter, and keeps the “Reroll” button

enabled for retry, and keeps the user on the Food Result screen with previously selected constraints unchanged.

5. Scenario 5: External API Failure

- GIVEN the user has an active internet connection but the Places API fails to respond or times out,
- WHEN the user taps "Reroll",
- THEN the system displays: "Something went wrong while finding your options. Please try again in a moment.", does not increment the reroll counter, keeps the "Reroll" button enabled for retry, and keeps the user on the Food Result screen with previously selected constraints unchanged.

6. Scenario 6: Reroll Unavailable - Single Match only

- GIVEN the initial search returned only one matching venue (no alternative available in the candidate list),
- WHEN the Food Result screen is displayed,
- THEN the "Reroll" button is disabled from the start, showing "No alternative available for this search".

US-03: Focus Workspace Matching

- User Story: As a student or remote worker, I want to select my current energy level and preferred work vibe, so that the system suggests a single tailored study/workspace location without decision fatigue.
- Acceptance Criteria:
 1. Scenario 1: Successful Spot Recommendation
 - GIVEN the user is on the Focus Input screen,
 - WHEN the user selects an Energy Level (Low, Medium, or High) and an Environment Vibe (Silent, Background Noise, or Collaborative), and taps the "Find My Spot" button,
 - THEN the system searches for locations matching the selected criteria, fetches full details for one randomly selected location from the list, and navigates to the Focus Result screen displaying exactly one matching location within 2.0 seconds, along with a weather forecast shown for user reference only and not factored into the matching logic.
 2. Scenario 2: Validation of Incomplete Input Selection
 - GIVEN the user is on the Focus Input screen,
 - WHEN the user attempts to tap "Find My Spot" without selecting both Energy and Vibe,
 - THEN the button remains disabled with a prompt requiring both selections.
 3. Scenario 3: Zero Matching Results
 - GIVEN no registered focus spaces match both criteria within range,
 - WHEN the user taps "Find My Spot",
 - THEN the system displays: "No spots found matching your criteria. Try adjusting your vibe or energy level." and return the user to the input selection screen with inputs retained.
 4. Scenario 4: Network Connectivity Lost
 - GIVEN the user is on the Focus Input screen with no active internet connection,
 - WHEN the user taps "Find My Spot",

- THEN the system displays: "No internet connection. Please check your network and try again.", keeps the user on the Focus Input screen with all selections retained, and keeps the "Find My Spot" button enabled for retry.
- 5. Scenario 5: Places API Failure
 - GIVEN the user has an active internet connection but the Places API fails to respond or times out,
 - WHEN the user taps "Find My Spot",
 - THEN the system displays: "Something went wrong while finding your options. Please try again in a moment.", keeps the user on the Focus Input screen with all selections retained, and keeps the "Find My Spot" button enabled for retry.
- 6. Scenario 6: Weather API Failure (Partial - Places API Succeeds)
 - GIVEN the user has an active internet connection and the Places API responds successfully, but the Weather API fails to respond or times out,
 - WHEN the user taps "Find My Spot",
 - THEN the system navigates to the Focus Result screen within 2.0 seconds, regardless Weather API failure, displaying the weather widget with "Weather unavailable".

US-04: Focus 1-Time Decision Reroll

- User Story: As a user dissatisfied with the initial suggestion, I want to reroll the recommendation once per session, so that I have a quick alternative option without getting trapped in endless decision-making.
- Acceptance Criteria:
 1. Scenario 1: Successful First Reroll
 - GIVEN the initial suggestion is displayed, the candidate list contains at least one more unfetched venue, and session reroll count equals 0,
 - WHEN the user taps the "Reroll" button,
 - THEN the system fetches details for the next venue in the candidate list and displays it within 2.0 seconds, increments the reroll counter to 1, and reuses the cached weather status from the initial search.
 2. Scenario 2: Enforcing Reroll Limit
 - GIVEN the user has already used their one allowed reroll in the active session,
 - WHEN the alternative suggestion is displayed,
 - THEN the "Reroll" button is disabled (greyed out) and displays a message: "No rerolls remaining for this session."
 3. Scenario 3: Resetting Reroll State on New Session
 - GIVEN a completed Focus session where a reroll occurred,
 - WHEN the user returns to the Home screen and initiates a fresh Focus request,
 - THEN the system resets the session reroll counter back to 0.
 4. Scenario 4: Network Connectivity Lost
 - GIVEN the user is on the Focus Result screen with no active internet connection suddenly,
 - WHEN the user taps "Reroll",
 - THEN the system displays: "No internet connection. Please check your network and try again.", does not increment the reroll counter, and keeps the "Reroll" button enabled for retry, and keeps the user on the Focus Result screen with previously selected constraints unchanged.

5. Scenario 5: Places API Failure

- GIVEN the user has an active internet connection but the Places API fails to respond or times out,
- WHEN the user taps "Reroll",
- THEN the system displays: "Something went wrong while finding your options. Please try again in a moment.", does not increment the reroll counter, keeps the "Reroll" button enabled for retry, and keeps the user on the Focus Result screen with previously selected constraints unchanged.

6. Scenario 6: Weather API Failure (Partial - Places API Succeeds)

- GIVEN the user has an active internet connection and the Places API responds successfully, but the Weather API fails to respond or times out,
- WHEN the user taps "Reroll",
- THEN the system generates an alternative option within 2.0 seconds, increments the reroll counter, displays the recommendation card and reuses the cached weather status from the initial search.

7. Scenario 7: Reroll Unavailable - Single Match only

- GIVEN the initial search returned only one matching venue (no alternative available in the candidate list),
- WHEN the Focus Result screen is displayed,
- THEN the "Reroll" button is disabled from the start, showing "No alternative available for this search".

US-05: Task Input & Weighted Matrix Priority Calculation

- User Story: As a busy individual, I want the system to calculate and rank my daily tasks based on importance and urgency, so that I know exactly which task to focus on first without feeling overwhelmed.
- Acceptance Criteria:
 1. Scenario 1: Automatic Task Ranking
 - GIVEN the user is on the Priority Input screen and enters at least two tasks with assigned Importance (High/Medium/Low) and Urgency levels,
 - WHEN the user taps "Rank My Tasks",
 - THEN the system applies the weighted scoring matrix and navigates to the Priority Queue screen displaying the single #1 top-priority task card within 1.0 second.
 2. Scenario 2: Validation for Insufficient Task Input
 - GIVEN the task list contains fewer than 2 active items (zero or one task),
 - WHEN the user attempts to tap "Rank My Tasks",
 - THEN the button remains disabled and displays a placeholder message: "Add at least 2 tasks to generate priorities."

US-06: Priority Task Completion & Queue Transition

- User Story: As a user completing my current task, I want to mark the active task as complete, so that the system logs my progress and automatically promotes the next highest-priority task in line.
- Acceptance Criteria:
 1. Scenario 1: Promoting the Next Task

- GIVEN the user has multiple ranked tasks and is viewing the #1 priority task,
 - WHEN the user taps “Mark as Done”,
 - THEN the system updates the task status to Completed, archives it to the Decision History, and renders the subsequent ranked task as the new active focus item.
2. Scenario 2: All Tasks Completed
- GIVEN the user marks the final remaining task in the queue as complete,
 - WHEN the user taps “Mark as Done”,
 - THEN the system renders a completion screen (“All clear! You’ve tackled all priority tasks for today.”) with an option to add new tasks.

US-07: User Authentication

- User Story: As a user, I want to register and log in securely, so that my preferences and decision history sync across sessions.
 - Acceptance Criteria:
1. Scenario 1: Successful Registration
- GIVEN the user is on the Registration screen and enters a valid email and a password meeting security requirements (minimum 8 characters, at least 1 letter and 1 number),
 - WHEN the user taps “Sign Up”,
 - THEN the system creates the account, logs the user in automatically, and navigates to the Home screen.
2. Scenario 2: Successful Login
- GIVEN the user is on the Login screen and enters a registered email and correct password,
 - WHEN the user taps “Log In”,
 - THEN the system authenticates the user and navigates to the Home screen within 2.0 seconds.
3. Scenario 3: Invalid Login Credentials
- GIVEN the user is on the Login screen,
 - WHEN the user enters an incorrect email/password combination and taps “Log In”,
 - THEN the system displays: “Incorrect email or password. Please try again.” and keeps the user on the Login screen.
4. Scenario 4: Validation on Incomplete Registration
- GIVEN the user is on the Registration screen,
 - WHEN the user attempts to tap “Sign Up” with an invalid email or a password not meeting the minimum requirements (8+ characters, 1 letter, 1 number),
 - THEN the system prevents submission and highlights the invalid fields with an inline prompt explaining the requirement not met.
5. Scenario 5: Network Connectivity Lost
- GIVEN the user is on the Login or Registration screen with no active internet connection,
 - WHEN the user taps “Log In” or “Sign Up”,
 - THEN the system displays: “No internet connection. Please check your network and try again.” and keeps the button enabled for retry.
6. Scenario 6: Authentication Service Failure
- GIVEN the user has an active internet connection but the authentication service (Firebase Auth) fails to respond or times out,

- WHEN the user taps “Log In” or “Sign Up”,
 - THEN the system displays: “Something went wrong. Please try again in a moment.” and offers a “Retry” action.
7. Scenario 7: Forget Password
- GIVEN the user is on the Login screen,
 - WHEN the user taps “Forget Password” and submits their registered email,
 - THEN the system sends a password reset link to that email and displays: “A reset link has been sent.”

US-08: Decision History Log

- User Story: As a user, I want to view a log of my past accepted decisions, so that I can reflect on my choices and see my Mental Energy dashboard trends.
 - Acceptance Criteria:
1. Scenario 1: Viewing Decision History
 - GIVEN the user has at least one accepted decision logged (from Food, Focus, or Priority),
 - WHEN the user navigates to the History screen,
 - THEN the system displays this week’s summary stats (total decisions, reroll rate, most active time), a module breakdown (Food/Focus/Priority), and a list of recent decisions with module icon, item name, and timestamp.
 2. Scenario 2: Empty History
 - GIVEN the user has no accepted decisions logged yet,
 - WHEN the user navigates to the History screen,
 - THEN the system displays an empty state message (“No decisions yet - start with Food, Focus, or Priority”).
 3. Scenario 3: History Load Failure
 - GIVEN the user navigates to the History screen with no active internet connection, or the cloud database fails to respond,
 - WHEN the screen attempts to load history data,
 - THEN the system displays: “Unable to load your history right now. Please check your connection and try again.” with a “Retry” action, instead of showing stale or partial data.

7. Key Design Considerations

Assumption and design decisions surfaced while writing the Acceptance Criteria, and collected here for a single reference point, in addition to the notes within each User Story.

1. Tie-break rule (Priority module)
 - Assumption: When two or more tasks receive an identical weighted score, the task entered first is ranked higher.
 - Impact: Without this rule, ranking order for tied tasks would be undefined/unstable, undermining user trust in the #1 recommendation. This is a proposed rule, not verified against the built app (need QA/dev confirmation before implementation).
2. Weather is informational only (Focus module)

- Assumption: Weather forecast is displayed on the Focus Result screen for user reference only and is not factored into the spot-matching algorithm matching is based on Energy Level and Environment Vibe.
- Impact: Keeping weather informational-only simplifies the MVP scope and avoids conflating two distinct concerns between matching logic and contextual display, but is worth revisiting as a Future Enhancement (Section 10) if user feedback shows demand for weather-aware matching.
- 3. Recommendations are filtered by operating hours (Food module)
 - Assumption: The system only surfaces venues that are open at the user's requested eating time: closed venues are excluded from the result pool entirely, rather than being recommended and then flagged as unavailable.
 - Impact: This "filter first" approach was chosen over "recommend then warn" because it better supports the app's core value proposition: a single, trustworthy recommendation the user does not need to double-check (see Elicited Requirement #1, Section 5). It depends on the Places API's operating-hours field being accurate and up to date, a dependency worth flagging if data quality issues arise later.
- 4. Manual Importance/Urgency self-assessment (Priority module)
 - Assumption: For the MVP, users manually tag each task's Importance and Urgency, meaning the system does not infer these values automatically.
 - Impact: This creates a potential tension with the app's core goal of reducing decision fatigue: asking an already-overwhelmed user to judge their own task priority may reintroduce the exact cognitive burden SwiftChoice is meant to remove. Raised as a discussion point by course instructor; addressed as a Future Enhancement (AI-assisted prioritisation, Section 10) rather than solved in the MVP, since no reliable proxy (like deadline-based Urgency inference) has been validated yet.
- 5. Two-step fetch: Search then Detail (Food/Focus reroll)
 - Assumption: The system performs a Search call once per request, then a Detail call for only the currently displayed venue. Reroll fetches Detail for the next candidate in the already retrieved list (not repeat the Search step).
 - Impact: This avoids the cost risk flagged. It also explains why Reroll can be instantly disabled when only 1 match exists and why API failure on Reroll is now scoped to the Details-fetch step only.

8. Requirements Traceability Matrix

Req ID	Business Goal Ref	User Story ID & Title	MoSCoW Priority	Technical / Component Layer	UAT Test Case ID	Status
REQ-01	Decision Fatigue Reduction	US-01: Food Meal Recommendation	Must Have	Places API (Eat Out) / Filter Engine / Local Recipe Pool (Eat In)	TC-FOO D-01a, TC-FOO D-01b,	Proposed

REQ-02		US-02: Food 1-Time Reroll Constraint	Should Have	Session State Manager	TC-FOOD-02	Proposed
REQ-03		US-03: Focus Workspace Matching	Must Have	Focus Matching Engine / Weather API (display only)	TC-FOC-01	Proposed
REQ-04		US-04: Focus 1-Time Reroll Constraint	Should Have	Session State Manager	TC-FOC-02	Proposed
REQ-05	Task Overload Relief	US-05: Task Input & Weighted Matrix Priority Calculation	Must Have	Weighted Scoring Algorithm	TC-PRI-01	Proposed
REQ-06		US-06: Task Completion & Queue Transition	Should Have	Local Storage/State Engine	TC-PRI-02	Proposed
REQ-07	Supporting Infrastructure	US-07: User Authentication	Must Have	Gateway Tier / Firebase Auth	TC-AUTH-01	Proposed
REQ-08		US-08: Decision History Log	Should Have	History & Analytics Service / Cloud Database	TC-HIST-01	Proposed

9. Process Diagram

Use Case Diagram

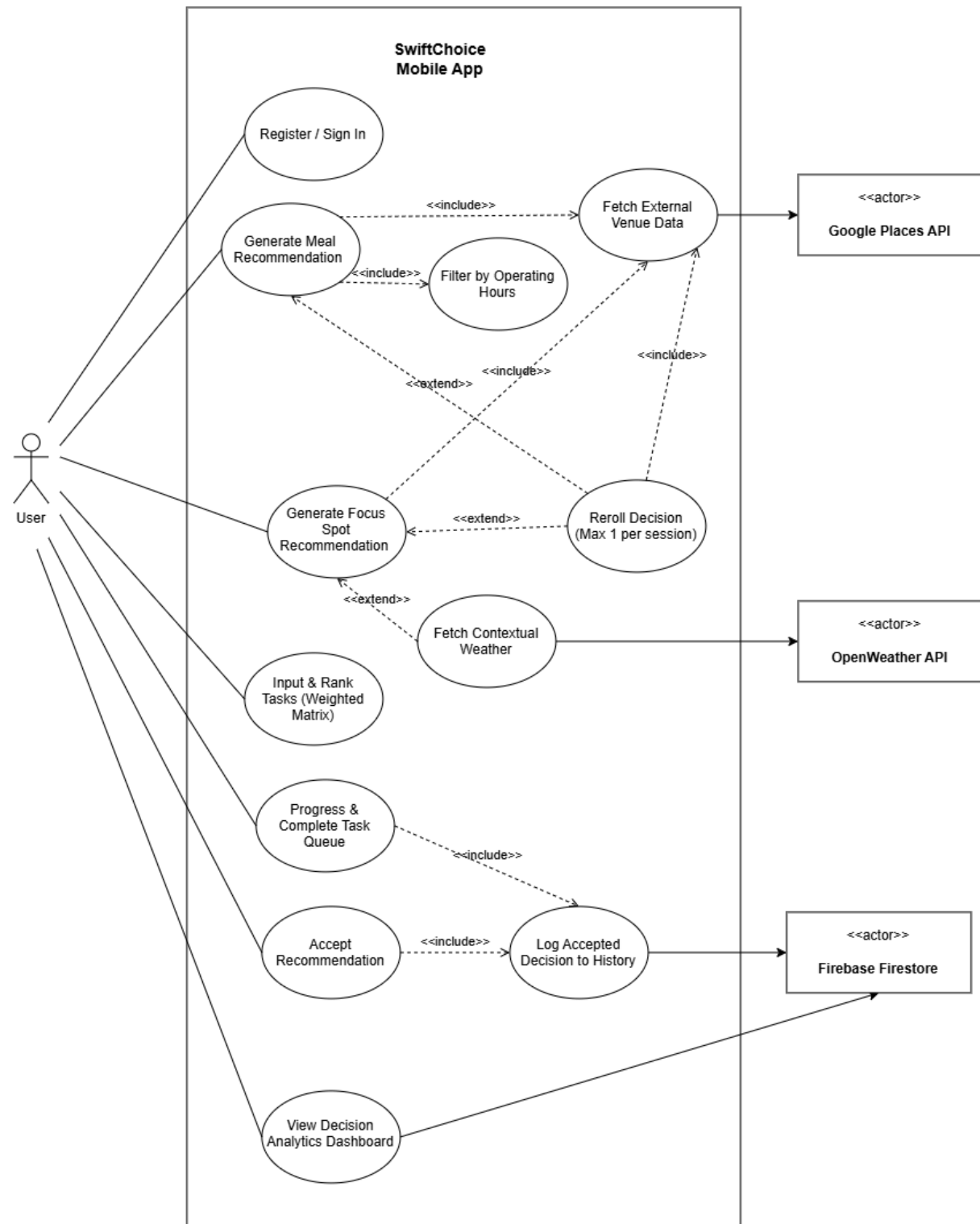


Figure 1. Use Case Diagram

Activity Diagram

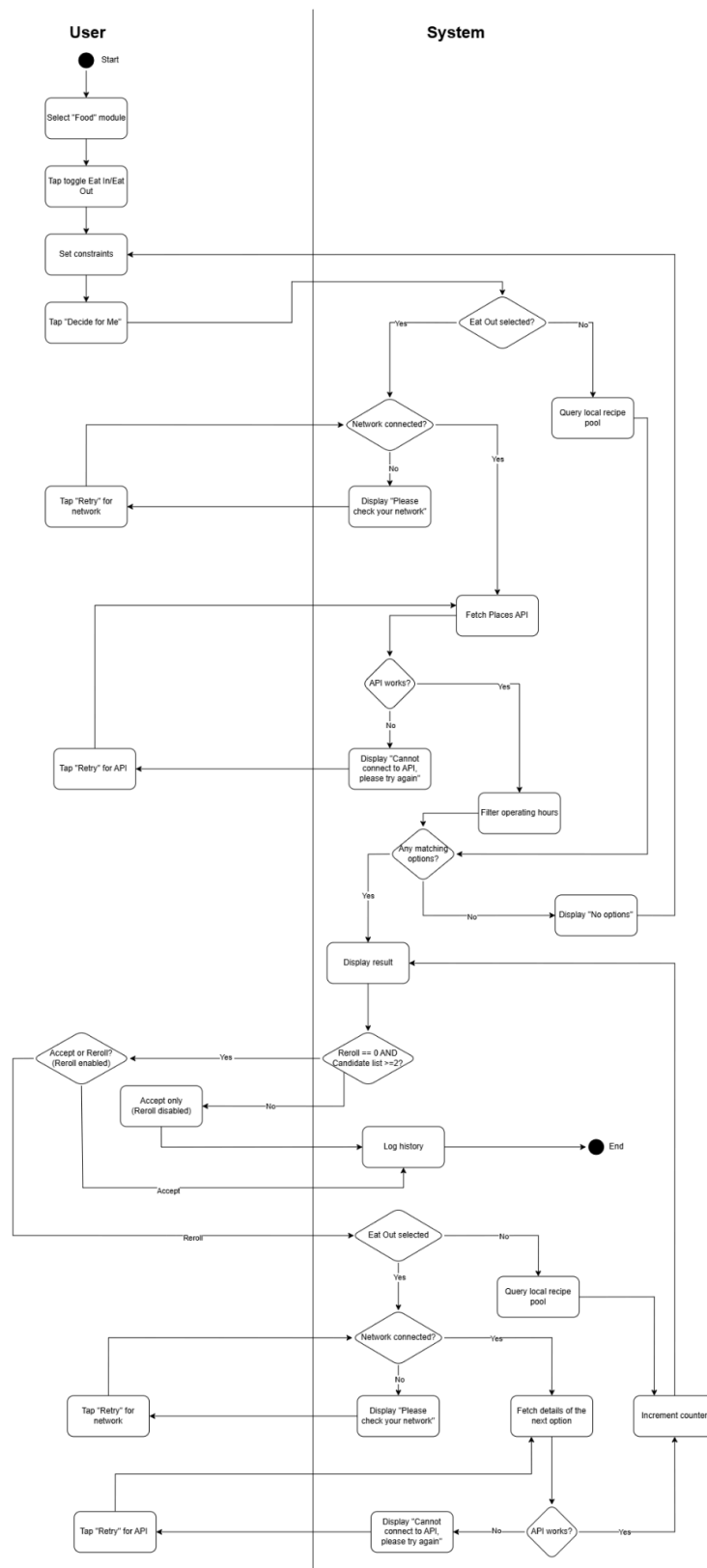


Figure 2. Activity Diagram: Food Module Workflow and Reroll Logic

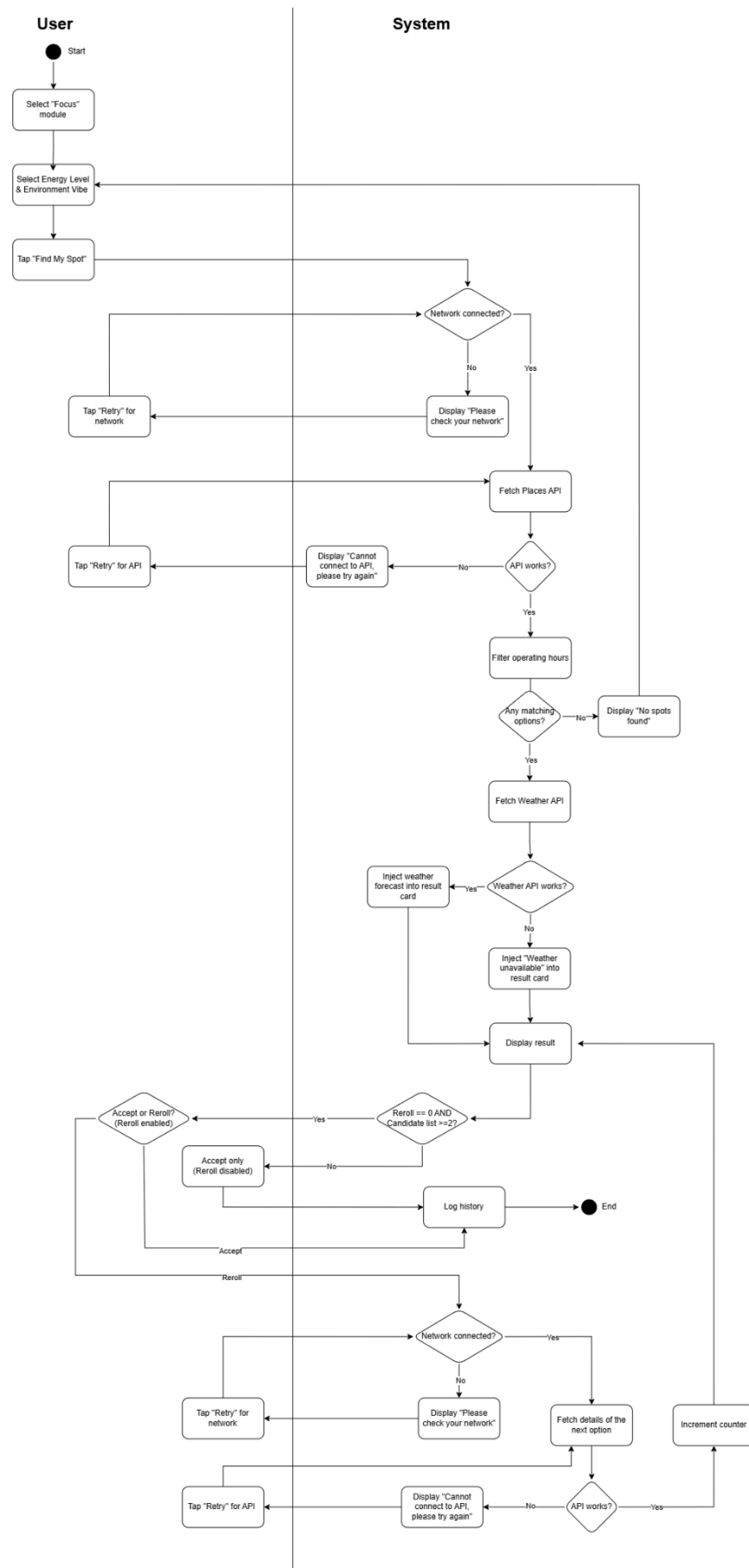


Figure 3. Activity Diagram: Focus Module Workflow and Reroll Logic

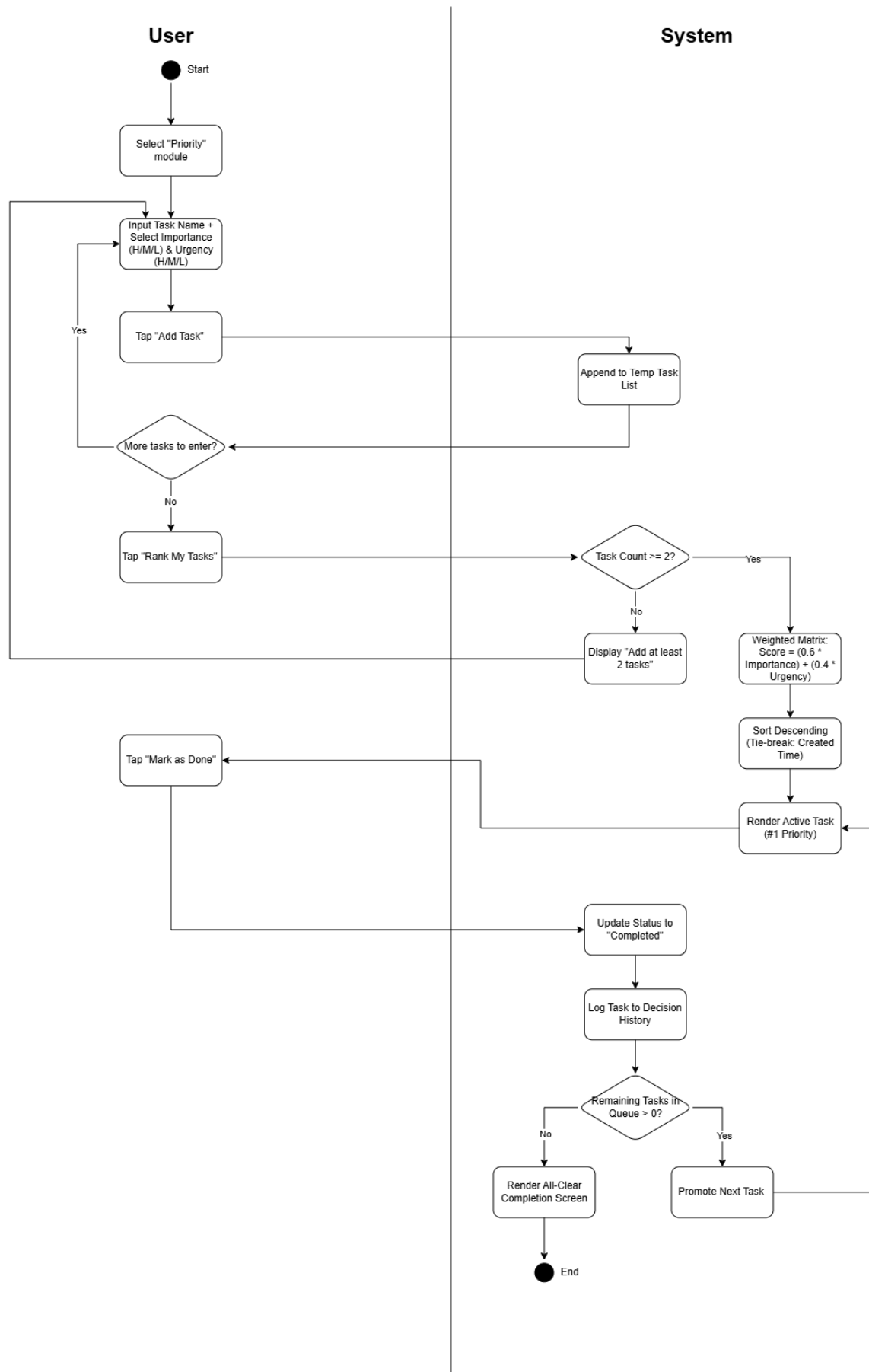


Figure 4. Activity Diagram: Priority Module Workflow

ER Diagram

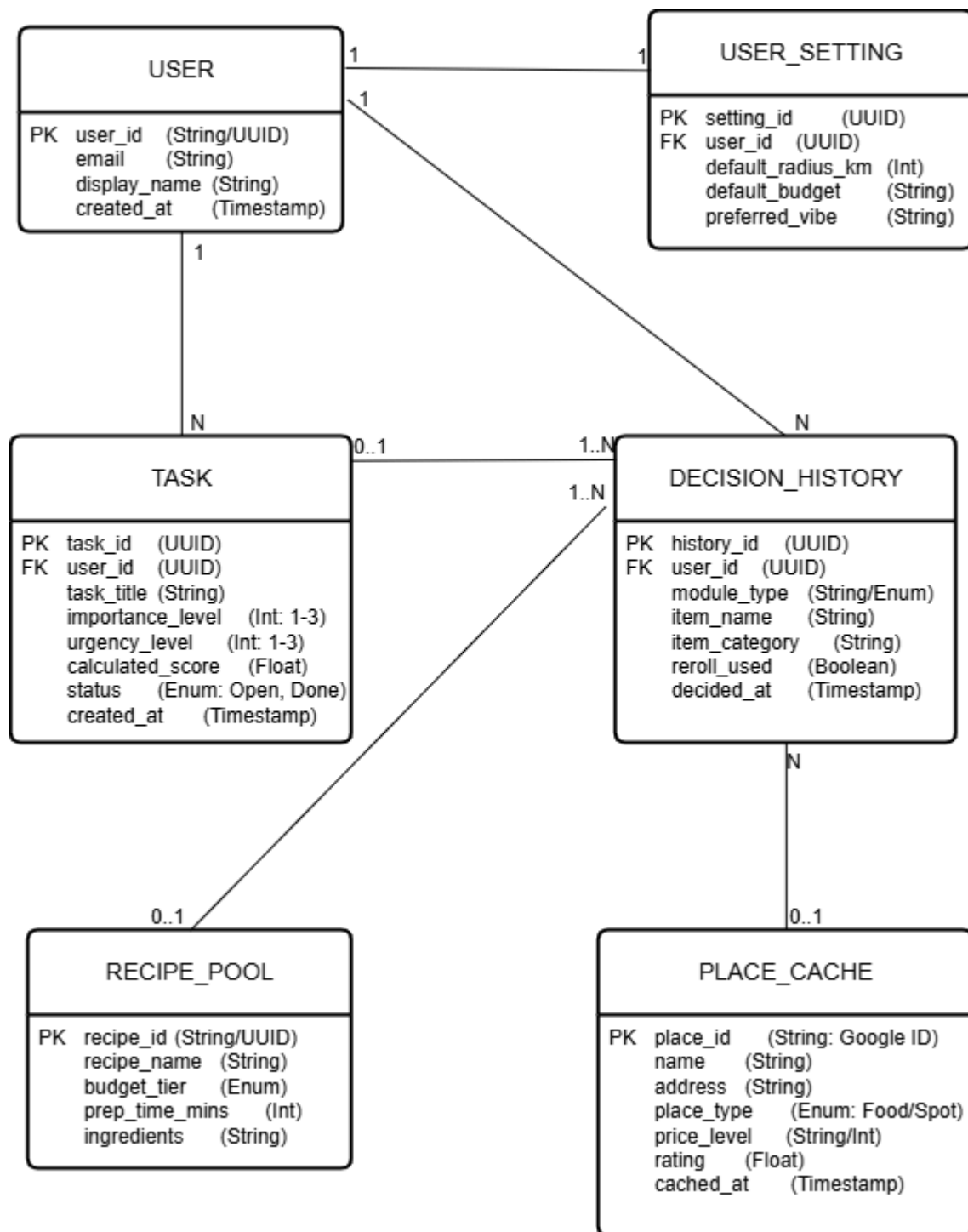


Figure 5. Entity-Relationship Diagram (ERD)

Data Dictionary

1. USER

Represent the registered mobile user profile, synchronised with the authentication provider.

Attribute Name	Data Type	Constraint	Nullable?	Description
user_id	String(UUID)	PK	No	Unique system identifier generated upon registration (matches Firebase Auth UID).
email	String(255)	Unique	No	Validated email address used for login and notifications.
display_name	String(100)	-	Yes	User's preferred name or alias displayed on the home screen.
created_at	Timestamp	-	No	UTC timestamp when the user account was provisioned.

2. USER_SETTING

Represent the registered mobile user profile, synchronised with the authentication provider.

Attribute Name	Data Type	Constraint	Nullable?	Description
setting_id	String(UUID)	PK	No	Unique identifier for the user preference record.
user_id	String(UUID)	FK	No	References USER.user_id (1:1 cardinality).
default_radius_km	Integer	Default: 5	No	Default geographic search radius for venue discovery.
default_budget	Enum	-	No	Default financial filter. Allow values: Budget, Moderate, Premium.
preferred_vibe	Enum	-	No	Default Focus environment setting. Allow values: Silent, Moderate, Lively.
updated_at	Timestamp	-	No	Timestamp of the most recent preference change.

3. TASK

Represent items managed within the Priority Module queue.

Attribute Name	Data Type	Constraint	Nullable ?	Description
task_id	String(UUID)	PK	No	Unique identifier for each created task item.
user_id	String(UUID)	FK	No	References USER.user_id (1:N cardinality)
task_title	String(255)	-	No	Description entered by the user (minimum 2 characters).
importance_level	Integer	Check (1-3)	No	User rating: Low (1), Medium (2), High (3).
urgency_level	Integer	Check (1-3)	No	User rating: Low (1), Medium (2), High (3).
calculated_score	Decimal(3,2)	-	No	Weighted score derived from: (0.6 * Importance) + (0.4 * Urgency).
status	Enum	Default: Open	No	Queue state: Open or Completed
created_at	Timestamp	-	No	Task creation timestamp; serves as the tie-breaker for identical scores.

4. PLACE_CACHE

Stores external venue metadata fetched from the Google Places API to minimize redundant third-party API calls and prevent latency.

Attribute Name	Data Type	Constraint	Nullable?	Description
place_id	String(255)	PK	No	External unique place identifier returned by Google Places API.
name	String(255)	-	No	Commercial name of the establishment.
address	String(500)	-	No	Human-readable formatted street address.
place_type	Enum	-	No	Categorizes the venue capability: Food (restaurant/cafes) or Focus (libraries/spaces).

price_level	Enum	-	Yes	Price tier indicator returned by external API. (mapped from Google Places API response: Level 0-1 -> Budget, Level 2 -> Moderate, and Level 3-4 -> Premium)
rating	Decimal(2,1)	Check (1.0-5.0)	Yes	Aggregate user rating from Google Places.
cached_at	Timestamp	-	No	Timestamp of external fetch; used for cache eviction.

5. RECIPE_POOL

A local catalog of quick meal options utilized by the user selects the “Eat In” path.

Attribute Name	Data Type	Constraint	Nullable?	Description
recipe_id	String(UUID)	PK	No	Unique identifier for the curated recipe.
recipe_name	String(255)	-	No	Title of the dish.
budget_tier	Enum	-	No	Cost bracket based on standard pantry ingredients: Budget, Moderate, Premium.
prep_time_mins	Integer	-	No	Estimated preparation and cooking time in minutes.
ingredients_summary	Text	-	No	List or markdown summary of key ingredients.

6. DECISION_HISTORY

Unified transaction log capturing every accepted recommendation or completed task across all three modules.

Attribute Name	Data Type	Constraint	Nullable?	Description
history_id	String(UUID)	PK	No	Unique event log identifier.
user_id	String(UUID)	FK	No	References USER.user_id (1:N cardinality).

module_type	Enum	-	No	Originating feature: Food, Focus, or Priority.
item_name	String(255)	-	No	Name of the accepted venue, recipe title, or completed task.
item_reference _id	String(255)	-	Yes	References place_id (Google), recipe_id, or task_id where applicable.
reroll_used	Boolean	Default: False	No	Indicates whether the user accepted the initial recommendation or used their reroll.
decided_at	Timestamp	-	No	Timestamp of acceptance

Sequence Diagram

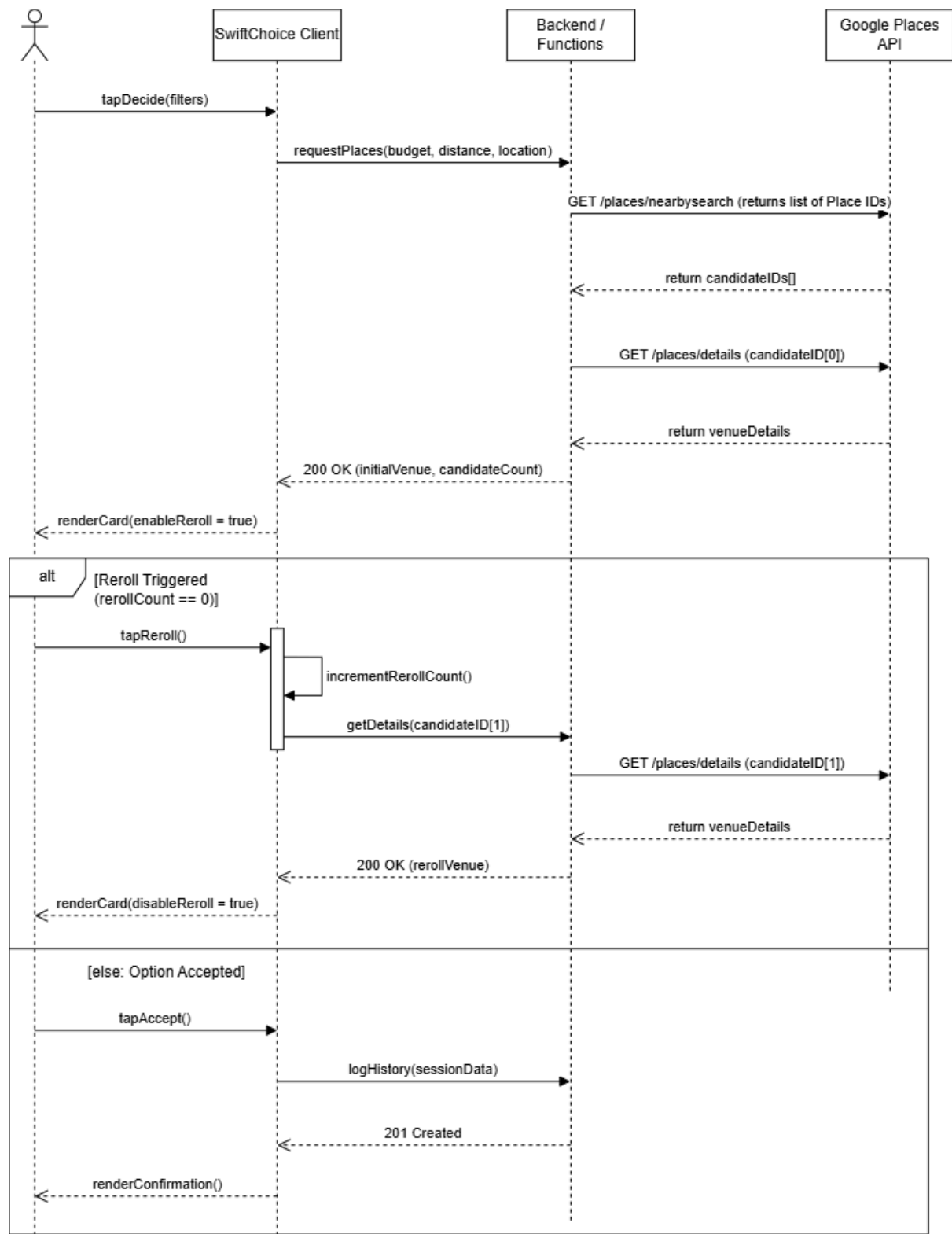


Figure 6. System Sequence Diagram (SSD): Places API Interaction & Reroll Flow

10. UAT Test Cases

Test Case: TC-FOOD-01a

- Requirement Ref: REQ-01 / US-01 (Scenario 1a)
- Test Title: Verify single meal recommendation (Eat Out) with valid filters, address display, and operating-hours filtering.
- Pre-conditions: User logged in; device GPS enabled; network connection active; "Eat Out" selected.
- Execution Steps:
 1. Open the Food module from the Home screen.
 2. Select "Eat Out"
 3. Select Budget < \$15, Prep Time < 20 mins, and Distance < 5 km.
 4. Tap the "Decide For Me" button.
- Expected Result: The app displays the Food Result screen within 2.0 seconds showing exactly 1 venue card with Name, Address, Price Tier, Distance, explanation label, and a filters summary line; the recommended venue is confirmed open at the requested time.
- Status: Proposed.

Test Case: TC-FOOD-01b

- Requirement Ref: REQ-01 / US-01 (Scenario 1b)
- Test Title: Verify single meal recommendation (Eat In) using the local recipe pool.
- Pre-conditions: User logged in; "Eat In" selected (no network dependency)
- Execution Steps:
 1. Open the Food module from the Home screen.
 2. Select "Eat In"
 3. Select Budget < \$15 and Prep Time < 20 mins.
 4. Tap "Decide for Me".
- Expected Result: The app displays the Food Result screen within 2.0 seconds showing exactly 1 recipe/meal card with Name, Price Tier, Prep Time, explanation label, and a filters summary line
- Status: Proposed

Test Case: TC-FOOD-02

- Requirement Ref: REQ-02 / US-02
- Test Title: Verify 1-time reroll constraint and button state locking.
- Pre-conditions: Initial meal recommendation card rendered; session_reroll_count = 0.
- Execution Steps:
 1. Tap the "Reroll" button on the recommendation screen.
 2. Confirm an alternative meal recommendation display.
 3. Check the UI state of the "Reroll" button.
 4. Attempt to tap the "Reroll" button a second time.

- Expected Result: A distinct alternative option renders within 2.0 seconds; session_reroll_count increments to 1; the "Reroll" button becomes disabled/greyed out with label "No rerolls remaining for this session"; further taps trigger no action.
- Status: Proposed.

Test Case: TC-FOC-01

- Requirement Ref: REQ-03 / US-03
- Test Title: Verify Focus workspace matching on Energy and Vibe inputs, with weather shown as informational only.
- Pre-conditions: User navigated to Focus module input screen; network connection active.
- Execution Steps:
 1. Select Energy Level: Medium.
 2. Select Environment Vibe: Silent.
 3. Tap "Find My Spot".
- Expected Result: Selections highlight; screen transitions within 2.0 seconds to the Focus Result screen showing exactly 1 matching location, with a weather forecast displayed for reference only.
- Status: Proposed.

Test Case: TC-FOC-02

- Requirement Ref: REQ-04 / US-04
- Test Title: Verify 1-time reroll constraint for Focus.
- Pre-conditions: Initial Focus recommendation card rendered; session_reroll_count = 0.
- Execution Steps:
 1. Tap the "Reroll" button on the Focus Result screen.
 2. Confirm an alternative location display.
 3. Check the UI state of the "Reroll" button.
 4. Attempt to tap the "Reroll" button a second time.
- Expected Result: A distinct alternative option renders within 2.0 seconds; counter increments to 1; the "Reroll" button becomes disabled/greyed out; further taps trigger no action.
- Status: Proposed.

Test Case: TC-PRI-01

- Requirement Ref: REQ-05 / US-05
- Test Title: Verify weighted matrix task sorting and top item rendering.
- Pre-conditions: Priority module open; task list empty.
- Execution Steps:
 1. Attempt to tap "Rank My Tasks" with 0 tasks entered.
 2. Enter Task 1: "Submit Report" (Importance: High, Urgency: High).
 3. Attempt to tap "Rank My Tasks" with only 1 task entered.
 4. Enter Task 2: "Check Emails" (Importance: Low, Urgency: Low).

5. Tap "Rank My Tasks".

- Expected Result: The button remains disabled with "Add at least 2 tasks to generate priorities" for both 0 and 1 task; once 2 tasks are present, the algorithm ranks Task 1 above Task 2 and navigates within 1.0 second to the Priority Queue displaying "Submit Report" as the active #1 card.
- Status: Proposed.

Test Case: TC-PRI-02

- Requirement Ref: REQ-06 / US-06
- Test Title: Verify task completion and automatic queue progression.
- Pre-conditions: Active Priority Queue has 2 tasks loaded; Task 1 displayed on screen.
- Execution Steps:
 1. Tap "Mark as Done" on Task 1.
 2. Verify screen update.
 3. Open Decision History tab.
- Expected Result: Task 1 transitions to Completed; Task 2 immediately promotes to the active focus card; Task 1 appears as a completed entry in the Decision History log.
- Status: Proposed

Test Case: TC-AUTH-01

- Requirement Ref: REQ-07 / US-07
- Test Title: Verify registration and login with valid credentials.
- Pre-conditions: No existing account for the test email.
- Execution Steps:
 1. Enter a valid email and password (8+ characters, 1 letter, 1 number) on the Registration screen.
 2. Tap "Sign Up".
 3. Log out
 4. Log back in with the same credentials.
- Expected Result: Account is created and user is auto-logged in on registration; login succeeds and navigates to Home within 2.0 seconds.
- Status: Proposed

Test Case: TC-HIST-01

- Requirement Ref: REQ-08 / US-08
- Test Title: Verify history display with and without prior decisions.
- Pre-conditions: User logged in.
- Execution Steps:
 1. With 0 decisions logged, navigate to the History screen.
 2. Accept 1+ decisions across modules (Food, Focus, Priority).
 3. Revisit the History screen.

- Expected Result: Empty state message shown when no decisions exist; summary stats, module breakdown, and recent decisions list render correctly once decisions are logged.
- Status: Proposed

11. Future Enhancements

- Weather-Aware Matching (Focus): Currently, weather forecast is displayed as informational context only and does not influence spot recommendations. A future iteration could incorporate weather into the matching algorithm.
- AI-Assisted Prioritisation (Priority): Currently, tasks ranking uses a static weighted scoring matrix (Importance, Urgency) requiring manual self-assessment from the user. A future iteration could integrate an LLM to infer urgency/importance from task descriptions, reducing the self-assessment burden.
- SwiftChoice Pro (Subscription Tier): Unlimited decision history, learned preferences based on past choices, and ad-free experience.